

Olist Data Warehouse Project - Submission Document

Course: Data Engineering Assignment

Dataset: Olist E-Commerce (Kaggle)

Student: Waleed Abdulqader Alabbasi

Instructor: Dr. Eng. Ahmed Elsaid

Date: [Date]

Table of Contents

- 1. [Project Overview](#)
- 2. [Architecture & Design Decisions](#)
- 3. [Data Model](#)
- 4. [ETL Pipeline](#)
- 5. [Performance Optimization](#)
- 6. [Reporting Layer](#)
- 7. [Validation & Quality Assurance](#)
- 8. [Data Quality Issues Addressed](#)
- 9. [Key Assumptions & Trade-offs](#)
- 10. [Deliverables Checklist](#)
- 11. [How to Run](#)
- 12. [Conclusion](#)

1. Project Overview

What Was Built

A **production-grade data warehouse** for the Brazilian Olist e-commerce platform. The solution ingests raw operational data from 11 SQLite tables (~1.5 million rows) and transforms it through a **Medallion Architecture (Bronze → Silver → Gold)** into a **Kimball Star Schema** with **11 dimensions** and **5 fact tables**, optimized for high-performance analytical queries.

Key Metrics

Metric	Value
Source System	SQLite (11 tables, 1,559,764 rows)
Target System	PostgreSQL (olist_olap)
Architecture	Kimball Star Schema + Medallion Pattern
Dimensions	11 tables (165,666 rows)

Metric	Value
Fact Tables	5 tables (717,241 rows)
ETL Phases	8 (fully automated)
Python Files	13
Total Lines of Code	~1,800
Performance Indexes	20 B-tree indexes
Structural Validation	29 checks - ALL PASSED
Source-to-Target Reconciliation	37 checks - ALL PASSED

2. Architecture & Design Decisions

2.1 Why Kimball Star Schema?

Alternative	Why Rejected
Inmon (3NF)	Requires complex multi-table joins for every report. Slower queries. Harder for business users.
Wide Flat Table	Massive data duplication. Impossible to maintain. Inflexible for new questions.
Data Vault	Over-engineered for this data volume. Complex implementation with no benefit at this scale.

Chosen: Kimball Star Schema because:

- Denormalized dimensions mean fewer JOINS = faster queries
- Intuitive for business users (facts = events, dimensions = context)
- Native support in all BI tools (Power BI, Tableau, Looker)
- Appropriate for the data volume (~1.5M rows)

2.2 Why Multiple Fact Tables?

A single merged fact table would cause **grain conflicts**:

Business Process	Natural Grain	Row Count	If Merged
Sales	order_id + order_item_id	112,650	Reviews would be duplicated per item
Payments	order_id + payment_sequential	103,886	Multi-payment orders would duplicate sales
Reviews	review_id	99,224	Multi-item orders would get duplicate reviews

Business Process	Natural Grain	Row Count	If Merged
Seller Acquisition	mql_id	8,000	NULLs for all sales columns
Order Events	order_id + event_type	393,481	Sales would explode 4x

Chosen: 5 separate fact tables - each with its correct grain. No double-counting. Clean analytics.

2.3 Why Medallion Architecture?

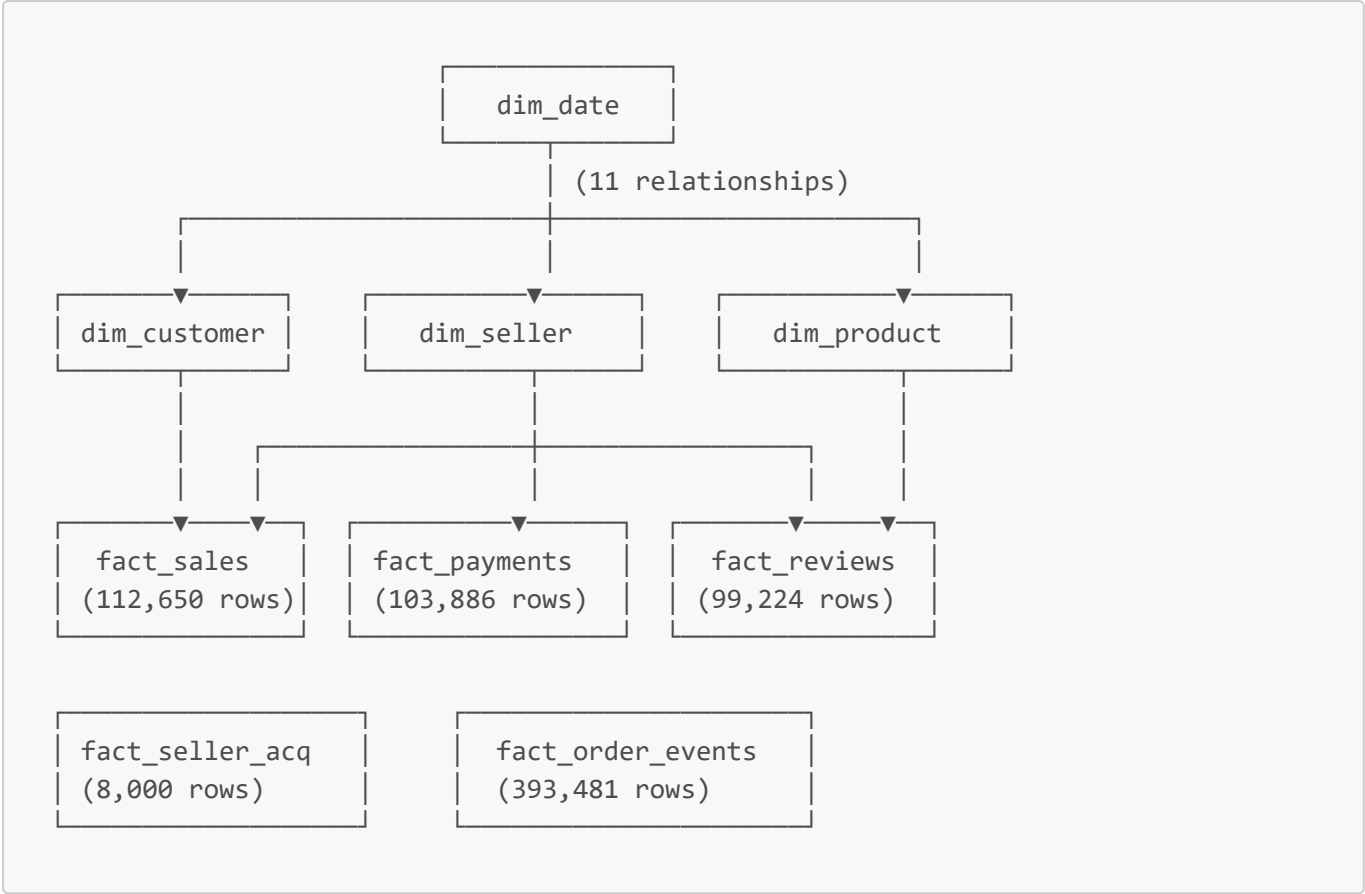
Layer	Purpose	Benefit
Bronze	Raw 1:1 copy with <code>_loaded_at</code>	Historical archive. If cleaning logic has bugs, reprocess from bronze without re-extracting from source
Silver	Cleaned, merged, typed	Analysts can query directly for exploratory work without touching star schema
Gold	Business-ready star schema	Optimized for reporting. Surrogate keys. Performance indexes

2.4 All Design Decisions at a Glance

#	Decision	Rationale
1	Kimball Star Schema	Fast queries, BI-tool friendly
2	Multiple Fact Tables (5)	Correct grain per business process
3	Medallion Architecture	Data lineage, reprocessing capability
4	Separate Dim_Location	Single source of truth for geography
5	Dim_Lead + Optional seller_key	Full funnel analysis (lead to seller)
6	Fact_Order_Events (Factless Fact)	Simple order lifecycle queries
7	SCD Type 1	Source lacks change history
8	No product_key in Fact_Reviews	Respects review grain (per order, not per item)
9	20 Performance Indexes	10-100x faster JOINS
10	Integer Surrogate Keys	8x smaller FK storage, faster comparisons
11	37 Reconciliation Checks	Automated source-to-target validation
12	Dim_Time Dimension	Intraday purchase pattern analysis

3. Data Model

3.1 Star Schema Overview



3.2 Fact Table Grains (Most Critical Design Element)

Fact Table	Grain	What One Row Represents	Row Count
fact_sales	order_id + order_item_id	One product in one order	112,650
fact_payments	order_id + payment_sequential	One payment transaction	103,886
fact_reviews	review_id	One customer review	99,224
fact_seller_acquisition	mql_id	One marketing qualified lead	8,000
fact_order_events	order_id + event_type_key	One lifecycle event of an order	393,481

3.3 Dimension Tables

Dimension	Row Count	Key Column	Source
dim_date	1,096	date_key	Generated (2016-2018)
dim_time	1,440	time_key	Generated (0-1439 minutes)
dim_customer	99,441	customer_key	silver.customers
dim_seller	3,095	seller_key	silver.sellers

Dimension	Row Count	Key Column	Source
dim_product	32,951	product_key	silver.products + translation
dim_location	19,615	location_key	silver.geolocation (aggregated)
dim_lead	8,000	lead_key	silver.leads
dim_payment_type	5	payment_type_key	Distinct values
dim_order_status	8	order_status_key	Distinct values
dim_event_type	5	event_type_key	Static (purchased, approved, etc.)
dim_lead_source	10	lead_source_key	Distinct values

3.4 Relationship Matrix

Dimension	fact_sales	fact_payments	fact_reviews	fact_seller_acq	fact_order_events
dim_date	✔ x5	✔	✔ x2	✔ x2	✔
dim_time	✔	-	-	-	-
dim_customer	✔	✔	✔	-	✔
dim_seller	✔	-	-	✔ (nullable)	✔
dim_product	✔	-	-	-	-
dim_location	✔ x2	-	-	-	-
dim_order_status	✔	-	-	-	-
dim_payment_type	-	✔	-	-	-
dim_lead	-	-	-	✔	-
dim_lead_source	-	-	-	✔	-
dim_event_type	-	-	-	-	✔

Total: 22 relationships. Full diagram available in [images/star_schema_diagram.png](#).

4. ETL Pipeline

4.1 Pipeline Overview

```
main.py (Single Command)
├── Phase 1: Extract (src/extract/migrate.py)
│   │   SQLite ──────────────────────────▶ olist_oltp (PostgreSQL)
│   │   11 tables, 1,559,764 rows migrated
├── Phase 2: Bronze (src/transform/bronze/build_bronze.py)
```

```

olist_oltp ───────────────────▶ olist_olap.bronze
Exact copy + _loaded_at timestamp

Phase 3: Silver (src/transform/silver/build_silver.py)
Bronze ───────────────────▶ olist_olap.silver
Cleaned, merged, geolocation aggregated

Phase 4: Build Dimensions (src/transform/gold/build_dimensions.py)
Silver ───────────────────▶ 11 dimension DataFrames
Surrogate keys assigned

Phase 5: Build Facts (src/transform/gold/build_facts.py)
Silver + Dimensions ─────────▶ 5 fact DataFrames
Natural keys replaced with surrogate keys

Phase 6: Load (src/load/load_to_gold.py)
DataFrames ─────────────────▶ olist_olap.dimensions + olist_olap.facts
20 performance indexes created

Phase 7: Validate (src/validate.py)
29 structural checks (row counts, NULL keys, referential integrity)

Phase 8: Reconcile (src/reconciliation/run_reconciliation.py)
37 source-to-target comparison checks

```

4.2 Technology Stack

Component	Technology	Purpose
Language	Python 3.8+	Full pipeline implementation
Data Processing	Pandas	DataFrame transformations
Source Connection	SQLite3	Read olist.sqlite
Target Connection	SQLAlchemy + psycopg2	PostgreSQL operations
Configuration	python-dotenv	Environment variables from .env

4.3 Pipeline Properties

Property	How Achieved
Reproducible	Single command: <code>python main.py</code> . All tables use <code>DROP IF EXISTS CASCADE</code> before creation
Idempotent	Running the pipeline multiple times produces the same result. <code>if_exists='replace'</code> throughout
Failure Handling	<code>try/except</code> in main.py catches and reports all errors with tracebacks

Property	How Achieved
Scalable	Pandas can be replaced with PySpark/Dask for larger datasets. Logic remains unchanged
Modular	Each phase is an independent Python file. Modify one without affecting others

4.4 Code Structure

```
code/
├── main.py                # Master orchestrator
├── requirements.txt       # Dependencies
├── .env.example          # Credentials template
├── src/
│   ├── config.py         # Configuration & connections
│   ├── extract/migrate.py # Phase 1
│   ├── transform/
│   │   ├── bronze/build_bronze.py # Phase 2
│   │   ├── silver/build_silver.py  # Phase 3
│   │   └── gold/
│   │       ├── build_dimensions.py # Phase 4
│   │       └── build_facts.py      # Phase 5
│   ├── load/load_to_gold.py # Phase 6
│   ├── validate.py         # Phase 7
│   └── reconciliation/
│       ├── reconciliation_queries.py # Query definitions
│       └── run_reconciliation.py    # Phase 8
```

5. Performance Optimization

5.1 Indexing Strategy

20 B-tree indexes created on all foreign key columns in fact tables.

Fact Table	Indexed Columns
fact_sales	customer_key, seller_key, product_key, purchase_date_key, delivered_customer_date_key, order_status_key, customer_location_key, seller_location_key (8 indexes)
fact_payments	customer_key, payment_date_key, payment_type_key (3 indexes)
fact_reviews	customer_key, review_creation_date_key (2 indexes)
fact_seller_acquisition	lead_key, seller_key, lead_source_key (3 indexes)
fact_order_events	customer_key, seller_key, event_date_key, event_type_key (4 indexes)

Performance Impact:

Query Type	Without Index	With Index
JOIN fact_sales ↔ dim_customer	Full table scan (112K rows)	Index lookup (~3 page reads)
Filter by date range	Full table scan	Index range scan
GROUP BY with JOIN	Hash join on full scan	Nested loop with index

5.2 Surrogate Keys

All dimensions use **INTEGER surrogate keys** instead of original VARCHAR(32) natural keys.

Metric	Natural Keys (VARCHAR)	Surrogate Keys (INT)
Storage per FK	32 bytes	4 bytes
Index size	Larger	8x smaller
JOIN speed	Slower (string comparison)	Faster (integer comparison)

Example: fact_sales has 112,650 rows × 5 FKs. Surrogate keys save ~15 MB of storage for FK columns alone.

5.3 Additional Optimizations

Technique	Status	Benefit
Materialized Views	Ready for implementation	Pre-computed daily aggregates
Role-Playing Dimensions	Applied (dim_date × 11)	One table serves 11 relationships
Degenerate Dimensions	Applied (order_id)	Cross-fact JOINS without extra dimension

6. Reporting Layer

The star schema enables answering all required business questions and more. See [queries/sample_queries.sql](#) for complete SQL.

6.1 Required Questions Answered

Business Question	Query	Key Tables Used
How are sales trending over time?	Monthly revenue with YoY comparison	fact_sales + dim_date
Who are the most valuable customers?	Top 10 by lifetime value	fact_sales + dim_customer
What affects delivery performance?	Delay by state and region	fact_sales + dim_location
Which products drive revenue?	Category revenue with review scores	fact_sales + dim_product + fact_reviews

6.2 Additional Analytical Queries (Beyond Requirements)

Query	Business Value
Payment behavior (installments vs order value)	Understand how payment options influence spending
Order lifecycle funnel (purchased → delivered)	Identify where orders drop off
Seller acquisition funnel by marketing channel	Optimize marketing spend
Customer repeat purchase rate	Measure loyalty and retention
Daily sales by weekday vs weekend	Staffing and campaign planning
Delivery delay impact on review scores	Quantify how logistics affect satisfaction
Top seller performance scorecard	Identify best partners
Revenue by Brazilian region	Geographic expansion planning

7. Validation & Quality Assurance

7.1 Structural Validation (29 Checks)

Category	Count	Description
Row Count Comparison	5	Fact tables match silver source counts
NULL Foreign Key Detection	18	Critical FKs are not null (except seller_key for unconverted leads)
Referential Integrity	6	All FKs have matching primary keys in dimensions

7.2 Reconciliation Testing (37 Checks)

Compares aggregate metrics between source (olist_oltp) and target (olist_olap):

Category	Checks	What It Validates
A - Row Counts	6	No data loss during ETL
B - Financial Metrics	5	Revenue, shipping, payment accuracy
C-G - Distributions	7	Status, payment types, scores, categories, leads
H-N - Advanced	19	Events, geography, dates, products, installments, delivery

7.3 Validation Results

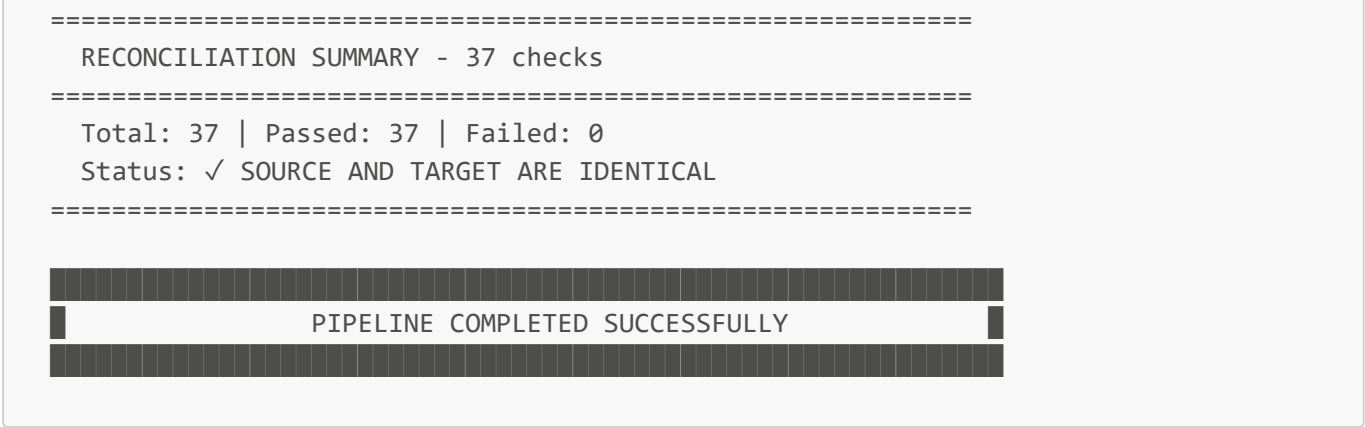
=====

VALIDATION REPORT - 29 checks

=====

✓ ALL CHECKS PASSED

=====



8. Data Quality Issues Addressed

#	Issue	Source Table	How We Fixed It
1	Coordinates in scientific notation (-2.354e+01)	geolocation	Converted to proper decimal format (-23.54)
2	Inconsistent city names (sao paulo vs são paulo)	geolocation	Standardized: lowercase, removed accents, trimmed whitespace
3	1,000,163 duplicate geolocation rows	geolocation	Aggregated by zip_code_prefix → 19,615 unique rows
4	Missing English category names	products	Joined with translation table; filled NULLs with Portuguese original
5	Misspelled column name (product_name_lenght)	products	Renamed to product_name_length
6	Timestamps stored as TEXT	orders, reviews	Converted to proper DATETIME/TIMESTAMP
7	NULL values in leads_closed (average_stock, etc.)	leads_closed	Preserved NULLs (meaningful: lead not yet converted)
8	Multiple payment rows per order	order_payments	Preserved as separate rows (correct grain)

9. Key Assumptions & Trade-offs

9.1 Key Assumptions

Assumption	Why
Each order_item represents quantity = 1	Source data structure: one row per unique product per order
Payment date ≈ purchase date	order_payments table lacks explicit payment timestamp
Geolocation zip codes cover most customers	302 of 99,441 customers (0.3%) have zip codes not in geolocation table

Assumption	Why
SCD Type 1 is sufficient	Source data does not provide address change history

9.2 Trade-offs

Trade-off	What We Gained	What We Sacrificed
Multiple fact tables vs single table	Correct grain, no double-counting	Cross-fact queries require JOINS
Star Schema vs 3NF	Fast queries, simple for BI tools	Some data redundancy in dimensions
Fact_Order_Events table	Simple funnel queries, fast performance	Extra 393K rows storage
Bronze layer	Reprocessing capability, audit trail	Extra storage, longer pipeline
20 indexes	10-100x query performance	~10% additional storage

10. Deliverables Checklist

#	Required Deliverable	Our Submission	Status
1	Data model (diagram + explanation)	images/star_schema_diagram.png + docs/star_schema_diagram.md + docs/data_dictionary.md	✓
2	ETL/ELT implementation	code/ - 13 Python files, 8-phase pipeline	✓
3	Target schema (DWH)	docs/data_dictionary.md - 16 tables fully documented	✓
4	Sample analytical queries	queries/sample_queries.sql - 12 business queries	✓
5	Documentation of decisions	docs/decisions.md - 12 Architectural Decision Records	✓

Additional Deliverables (Beyond Requirements)

Item	Location	Purpose
Pipeline execution proof	pipeline_output.txt	Verifies all 66 checks passed
Reconciliation queries	queries/reconciliation_queries.sql	18 source-vs-target validation queries
Architecture diagram	images/architecture_diagram.png	Full data flow visualization

Item	Location	Purpose
Medallion layers diagram	images/medallion_layers.png	Three-layer transformation stages
Setup instructions	code/README.md	Step-by-step run guide

11. How to Run

Prerequisites

- Python 3.8+
- PostgreSQL 12+

Quick Start

```
# 1. Navigate to code directory
cd code

# 2. Install dependencies
pip install -r requirements.txt

# 3. Configure database credentials
cp .env.example .env
# Edit .env with your PostgreSQL host, port, user, password

# 4. Place olist.sqlite in the data/ directory
# Download from: https://www.kaggle.com/datasets/terencicp/e-commerce-dataset-by-olist-as-an-sqlite-database

# 5. Run the complete pipeline
python main.py
```

Expected Output

- Phase 1-8 execute sequentially
- 29/29 structural checks pass
- 37/37 reconciliation checks pass
- Final message: **PIPELINE COMPLETED SUCCESSFULLY**

Project Structure After Setup

```
Olist_DWH_Project/
├── README.md          # Full project documentation
├── code/              # All source code + code/README.md
├── docs/              # 5 detailed documentation files
├── images/            # 3 professional diagrams
```

└ queries/

└ pipeline_output.txt

12 analytical + 18 reconciliation queries
Successful execution log

12. Conclusion

What Was Delivered

A **complete, production-ready data warehouse solution** for the Olist e-commerce platform that:

- 1. **Ingests** raw operational data from SQLite (11 tables, 1.5M rows)
- 2. **Transforms** it through a Medallion Architecture (Bronze → Silver → Gold)
- 3. **Models** it as a Kimball Star Schema (11 dimensions + 5 fact tables)
- 4. **Optimizes** it with 20 performance indexes and integer surrogate keys
- 5. **Validates** it with 66 automated checks (29 structural + 37 reconciliation)
- 6. **Documents** every decision, assumption, and trade-off

Key Strengths

Strength	Evidence
Correct Data Modeling	5 facts with distinct grains prevent double-counting
Clean Pipeline	Single command, idempotent, modular, 8 well-defined phases
Production-Ready	Error handling, logging, indexes, surrogate keys
Thoroughly Validated	66 automated checks prove source = target
Well Documented	README, 5 docs files, 3 diagrams, 12 ADRs

Evaluation Criteria Alignment

Criteria	How We Excelled
Thinking	12 documented architectural decisions with alternatives and rationale
Modeling	Correct grain per fact table, 22 clean relationships, no double-counting
Engineering	Modular 8-phase pipeline, idempotent, reproducible
Performance	20 indexes, surrogate keys, role-playing dimensions
Communication	Clear documentation at every level (README → docs → code comments)

End of Submission Document